

Regression Assignment2

Gabriel Severiche · 18 de septiembre de 2026



Contenido

Khipus.ai

Introduction to Machine Learning

Supervised Learning - Linear Regression

Assignment 2

Name : Gabriel Severiche

Assignment Instructions

1. Import the Titanic dataset
 2. Data Exploration
 3. Data Cleaning (addressing missing values)
- Feature Selection
5. Splitting Training and Test Data
 6. Use a linear regression model to predict the survivors
- Train the Model
- Evaluate the Model

Khipus.ai

Introduction to Machine Learning

Supervised Learning - Linear Regression

© Copyright Notice 2025, Khipus.ai - All Rights Reserved.

Assignment 2

Name : Gabriel Severiche

Assignment Instructions

Using the Titanic dataset provided:

1. Import the Titanic.csv file into a pandas dataframe
2. Perform a detailed data exploration.
3. Clean the dataset by addressing missing values.
4. Select relevant features for predicting survival.
5. Split the dataset into training and testing sets (80%-20% split).
6. Use a linear regression model to predict the survivors

1. Import the Titanic dataset

```
In [1]:  
  
# Import necessary libraries  
import pandas as pd  
import numpy as np  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error  
from sklearn.metrics import r2_score  
from sklearn.impute import SimpleImputer
```

```
In [2]:
# Your code here

# Load the Titanic dataset

data = pd.read_csv('titanic.csv')
```

2. Data Exploration

Before working with the dataset, it's important to understand its structure, data types, and summary statistics. Below are examples of how to explore the data.

```
In [3]:
# Your code here

# Display the first few rows of the dataset
# ...existing code...

data.head()
```

Out [3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helg...	female	22.0	1	1	3101298	12.2875	NaN	S

```
In [4]:
# Display summary statistics
data.describe()
```

Out [4]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	418.000000	418.000000	418.000000	332.000000	418.000000	418.000000	417.000000
mean	1100.500000	0.363636	2.265550	30.272590	0.447368	0.392344	35.627188
std	120.810458	0.481622	0.841838	14.181209	0.896760	0.981429	55.907576
min	892.000000	0.000000	1.000000	0.170000	0.000000	0.000000	0.000000
25%	996.250000	0.000000	1.000000	21.000000	0.000000	0.000000	7.895800
50%	1100.500000	0.000000	3.000000	27.000000	0.000000	0.000000	14.454200
75%	1204.750000	1.000000	3.000000	39.000000	1.000000	0.000000	31.500000
max	1309.000000	1.000000	3.000000	76.000000	8.000000	9.000000	512.329200

```
In [5]:
# Check for missing values
data.isnull().sum()
```

```
Out [5]:
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age           86
SibSp          0
Parch          0
Ticket         0
Fare           1
Cabin        327
```

```
Embarked      0
dtype: int64
```

3. Data Cleaning (addressing missing values)

Real-world datasets often contain missing values, duplicate rows, or incorrect data. Cleaning the data ensures its quality and usability for analysis.

```
In [6]:
data_cleaned = data.copy()

# Fill missing 'Age' with the median
data_cleaned['Age'] = data_cleaned['Age'].fillna(data_cleaned['Age'].median())

# Fill missing 'Embarked' with the mode
data_cleaned['Embarked'] = data_cleaned['Embarked'].fillna(data_cleaned['Embarked'].mode()[0])

# Drop 'Cabin' as it has too many missing values
data_cleaned = data_cleaned.drop('Cabin', axis=1, errors='ignore')
# Your code here

# Confirm there are no missing values left
data_cleaned = data_cleaned[data_cleaned.isnull().sum(axis=1) == 0]
data_cleaned.isnull().sum()
```

```
Out [6]:
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Embarked         0
dtype: int64
```

Feature Selection

To build an effective machine learning model, selecting relevant features is crucial. Select relevant features for predicting survival.

```
In [7]:
# Perform one-hot encoding on 'Sex' and 'Embarked' columns
# Define the features you want to use
# Select features and target
# Your code here
# Select the features from the DataFrame
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
target = 'Survived'
# Select the target variable from the DataFrame
data_selected = data_cleaned[features + [target]]
data_selected.head()
```

Out [7]:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
0	3	male	34.5	0	0	7.8292	Q	0
1	3	female	47.0	1	0	7.0000	S	1
2	2	male	62.0	0	0	9.6875	Q	0
3	3	male	27.0	0	0	8.6625	S	0
4	3	female	22.0	1	1	12.2875	S	1

5. Splitting Training and Test Data

```
In [15]:  
  
# Your code here  
# Split into training and test sets  
X=data_selected[features]  
y=data_selected[target]
```

6. Use a linear regression model to predict the survivors

When working with datasets, it's common to encounter missing values. These missing values can cause issues with many machine learning algorithms, which often require complete data to function correctly. To handle missing values effectively, we can use an imputer. The provided code demonstrates how to use the SimpleImputer from the sklearn.impute module to replace missing values with the mean of the respective column.

```
In [16]:  
  
# Convert categorical features to numeric dummy variables  
X = pd.get_dummies(data_selected[features], columns=['Sex', 'Embarked'], drop_first=True)  
y = data_selected[target]  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
# Instantiate the imputer with the strategy to replace missing values with the mean of the column  
imputer = SimpleImputer(strategy='mean')  
  
# Fit the imputer on the training data and transform it, replacing missing values with the mean  
X_train_imputed = imputer.fit_transform(X_train)  
  
# Transform the test data using the same imputer, ensuring consistency in handling missing values  
X_test_imputed = imputer.transform(X_test)
```

Train the Model

We will train a linear regression model using the training data.

```
In [17]:  
  
# Initialize the model  
model = LinearRegression()  
# Train the model  
model.fit(X_train_imputed, y_train)
```

Out [17]:

```
▼ LinearRegression ⓘ ?  
  
► Parameters  
  
► Fitted attributes
```

Evaluate the Model

We'll calculate the mean squared error to evaluate the model performance.

```
In [20]:  
  
# Make predictions  
model_predictions = model.predict(X_test_imputed)
```

```
# Evaluate the model
mse = mean_squared_error(y_test, model_predictions)
r2 = r2_score(y_test, model_predictions)
print(f'Mean Squared Error: {mse}')
print(f'Root Mean Squared Error: {np.sqrt(mse)}')
print(f'R-squared: {r2}')
```

Out [20]:

```
Mean Squared Error: 4.4035929623665484e-30
Root Mean Squared Error: 2.098473960373716e-15
R-squared: 1.0
```